

5 技术栈对比报告（含微前端架构）

实现日期：2026-06-30

功能：Epic Labs 复杂 demo（3关/4星型：drag-fill+flip-match+hotspot+quiz/Modal栈/localStorage/彩带/badge）

测试基准：Playwright 统一测试 9 项检查，全部 5 栈 9/9 通过，0 console error

1. 实验设计

对比维度	方法	对象
框架差异（React vs Vue SFC）	Method B：单次快照 + 代码结构分析	① vs ②
组件库差异	Method A：单次质量快照	① vs ③ vs ④

5 个栈：

- ① React 19.2 + Tailwind v4 + shadcn/ui（copy-in）
- ② Vue 3.5 + Tailwind v4 + shadcn-vue（copy-in）
- ③ React 19.2 + MUI 7（runtime CSS-in-JS）
- ④ React 19.2 + Ant Design 5（runtime CSS-in-JS）
- ⑤ React 19.2 + Tailwind v4（Module Federation 微前端，基于①按关卡拆分）

2. 核心指标一览

5 个栈在 Bundle 体积、源码行数、AI 文档支持三个维度的综合表现（⑤ MF 详细架构数据见第 8 节）：

!4技术栈核心指标一览

图：① React+Tailwind 在 LOC 和可维护性上综合最优；② Vue+Tailwind 体积最小；④ AntD AI 文档支持反而最好。（图为①-④ SPA 对比，⑤ MF 见第 8 节）

栈	LOC	Bundle gzip	AI 文档	架构
① react-shadcn	662	63 KB	✓	SPA
② vue-shadcn	732	28 KB	✓	SPA
③ react-mui	904	106 KB	✗	SPA
④ react-antd	1184	146 KB	✓✓	SPA
⑤ react-shadcn-mf	920 (+39%)	129 KB 首屏 (+105%)	✓	微前端 MF

3. Bundle 体积对比

!5栈Bundle体积对比

图：② vue-shadcn 体积最小（28KB）；⑤ MF 首屏 129KB 偏高（preview 模式 React 未共享），全量加载 258KB；生产部署 singleton 生效后首屏估算 ~20KB。

栈	JS gzip	相对①	架构
① react-shadcn	63 KB	基准	SPA
② vue-shadcn	28 KB	-55%	SPA
③ react-mui	106 KB	+68%	SPA
④ react-antd	146 KB	+132%	SPA
⑤ react-shadcn-mf（首屏）	129 KB	+105%	微前端（shell+level1）
⑤ react-shadcn-mf（全部加载）	258 KB	+310%	微前端（3关全加载）

关键发现：

- Vue3 框架本身比 React 19 轻约 35%，Tailwind 树摇激进，vue-shadcn 体积最小。
- MUI 和 AntD 包含 runtime CSS-in-JS（emotion/cssinjs），无法像 Tailwind 一样在编译期消除。
- AntD 146 KB 是 Tailwind 栈的 2.3 倍，主因是完整组件 + 样式 runtime。
- MF 版首屏 129 KB 偏高，因 vite preview 静态模式下 React 未真正共享；生产部署 React singleton 生效后首屏估算降至 ~20 KB。

4. 源码行数对比

!5栈源码行数对比

图：① react-shadcn 662 LOC 最精简；④ react-antd 1184 LOC 最多；⑤ MF 总计 920 LOC（+39%），其中 shell 339 + 3关合计 581；MF 增量主要来自类型定义重复和中间层文件。

栈	App/Shell	组件	data+lib	合计	架构
① react-shadcn	145	368	149	662	SPA
② vue-shadcn	175	413	145	733	SPA
③ react-mui	143	576	185	904	SPA
④ react-antd	237	622	274	1183	SPA
⑤ react-shadcn-mf	339 (shell)	367 (3关合计)	214	920	微前端 MF

关键发现：

- ① vs ②：Vue SFC 组件单文件 LOC 稍多 (+11%)。<template> 比 JSX 略啰嗦，但差距不大。
- ① vs ③：MUI 组件 LOC +37%。sx prop 内联样式比 Tailwind 类名冗长，3D 翻牌必须用内联 style。
- ① vs ④：AntD 组件 LOC +69%。AntD 无布局组件，ModalStack 需纯 div 实现。
- ① vs ⑤：MF 版 LOC +39%。增量来自类型定义跨子应用重复、新增 LevelApp 中间层、Shell 与子应用数据契约 (LEVEL_METAS) 冗余。

5. 框架结构差异 (① vs ②)

!框架结构对比

图：React 和 Vue 组件树结构几乎对称，主要差异在 App 层的状态管理写法。

React+Tailwind (①) 特点：纯函数组件 + JSX，状态逻辑与渲染在同一作用域，Tailwind 类名即文档。

Vue SFC (②) 特点：<script setup> + <template> 三段式，watchEffect + ref 比 useEffect + useState 稍多模板代码。

⚠ 单次快照的局限：Vue SFC "AI 持续追加导致文件膨胀"的命题尚未被本数据证实，需要 Method B 迭代实验验证。

6. AI 开发支持能力

!AI开发支持对比

图：MUI AI 文档评级最低 (D)，AntD 凭 6 个 llms.txt + MCP 工具反而评级最高 (A)。

反常之处：AntD 虽然 bundle 最大、LOC 最多，但 AI 文档支持最好——对重度依赖 AI 辅助的团队是真实优势。

7. 综合评分

!5栈综合评分

图：① 综合最优（AI时代首选），② 体积最优，③④ 适用于企业规范场景，⑤ MF 适合多团队/大规模部署场景，小项目架构开销大于收益。

8. 架构对比：SPA vs 微前端（⑤ react-shadcn-mf）

基于 react-shadcn 拆出的 Module Federation 版本（shell + level1/2/3 共 4 个子应用），实现相同功能，Playwright 9/9 通过。

!MF架构示意图

图：Shell 持有全局状态并通过 props 向下传递，3 个 remote 子应用各自只负责对应关卡的星型组件，无自身状态。

8.1 LOC 对比

维度	SPA react-shadcn	MF react-shadcn-mf	变化
总 LOC	662	920	+39%
App 层（shell）	145（App.tsx）	339（shell/src 全部）	+134%
组件层	368	367（3关组件之和）	≈持平
数据+lib	149	214（shell+3关数据之和）	+44%

LOC 增加的原因：类型定义在各子应用中重复（Star/DragFillConfig 等），中间层 LevelApp.tsx 是新增文件，Shell 与子应用间数据契约（LEVEL_METAS）带来冗余。

8.2 Bundle 对比

维度	SPA	MF（首屏 = shell + level1）	MF 理想值（react 真正共享后）
首屏 JS gzip	63 KB	129 KB（+105%）	~20 KB（-68%）
总计 JS gzip	63 KB	258 KB（+310%）	~30 KB（-52%）

说明：当前测量值偏高，因为 vite preview 静态模式下各子应用各自携带了完整的 react + react-dom（各 ~57KB）。生产部署（webpack/CDN 模式）中 React singleton shared 会生效，首屏体积可降至约 20 KB，按需加载后续关卡各约 3-5 KB。

8.3 AI 可维护性对比

场景	SPA	MF
定位一个 bug	在单仓库搜索，AI 有完整上下文	需先判断 bug 在 shell 还是哪个 level，上下文边界更清晰但需要额外一步
修改星型组件	直接改 src/components/DragFillStar.tsx	改 level1/src/components/DragFillStar.tsx，隔离清晰，但文件路径更深
追踪状态流	所有状态在 App.tsx，一个文件看完	Shell 持有全局状态，props 传入子应用，需跨文件追踪 props 接口
AI 指令精度	"修改 level2 的翻牌逻辑" → AI 直接找到文件	同样精确，子应用隔离让 AI 上下文更小

8.4 微前端适用场景分析

本次实验规模（3关，~900 LOC）对微前端来说**太小**——架构开销比业务代码本身还重。微前端的价值体现需要满足：

- 多团队独立开发部署（≥3 个团队）
- 子应用体积各自 > 100 KB（共享 React 才有规模收益）
- 明确的业务边界，不需要频繁跨子应用共享组件

对于 AI 时代的小型/中型项目：SPA 仍是更优解——代码更少、上下文更连续、AI 修改路径更短。微前端引入的类型重复、props 接口、构建复杂度，在 AI 辅助下不是"透明成本"，而是实际的维护负担。

9. 结论

最适合 AI 时代：① React + Tailwind + shadcn/ui

- 最精简的代码量（662 LOC），纯函数组件 + Tailwind 类名让 AI 生成的每一行可独立理解
- 合理的 bundle（64 KB gzip），shadcn copy-in 策略：组件代码在仓库里，AI 可直接读/改

框架选择（① vs ②）：单次快照 Vue SFC 仅多 11% LOC，体积更小，不应被轻易否定。迭代膨胀命题需专项测量。

组件库选择（③④）：若业务必须用 MUI/AntD，代价是 +40%~+79% LOC 和 +69%~+134% 体积；AntD AI 文档在大模型支持上反而优于 MUI。

架构选择（SPA vs MF）：小型项目 MF 带来 +39% LOC 和更高构建复杂度，收益需在多团队/大规模场景下才能显现。AI 时代应首选 SPA，在团队规模/部署需求明确触发后再引入微前端。

*详细数据见 docs/experiment-reports/measurements/，测试脚本见 demo/_test/test.mjs *

*微前端实现见 `demo/react-shadcn-mf/`，启动方式见 `start-preview.sh` *
